

Name: Sadam Hussain

SE-3A

Group members: Sadam Hussain & Muhammad Huzaifa

Roll No: 24p-3016

Roll No: 24p-3123

Assignment # 02

Project overview

The objective of this assignment was to modernize a legacy C++ loan calculator application. The original codebase suffered from poor organization (monolithic files), lack of input validation, potential integer overflows, and missing documentation.

Our team successfully refactored the project by restructuring the directory layout, implementing robust error handling, and switching data types to `long double` to support high-value financial calculations. Furthermore, we integrated **GoogleTest** for automated unit testing and **Doxygen** for generating technical documentation, ensuring the software is maintainable and reliable.

Refactoring & Bug Fixes

We performed several key technical improvements to meet the requirements:

The original single-file structure was split into a modular architecture to separate concerns. The source code was moved to a **src/** directory, and tests were isolated in **test/**.

- **Loan.h**
Contains the class definitions and Doxygen comments
- **Loan.cpp**
Contains the mathematical implementation of the loan logic.
- **Main.cpp**
Handles user input, command-line arguments, and configuration file reading.

. Bug Fix: Integer Overflow & Precision

The original code used **int** and **float** which caused inaccuracies for large loan amounts or long tenures. We replaced these with **long double** throughout the application to ensure precision.

Code snippet

Loan.cpp

```
long double LoanCalculator::calculatePayment() { if (periodTotal_ == 0) return 0;

// Converted to long double for higher precision
long double monthlyInterest = interest_ / 1200.0;
long double principal = amount_ - initialPayment_;

// Standard EMI Formula:  $P * r * (1+r)^n / ((1+r)^n - 1)$ 
return (principal * monthlyInterest * pow(1 + monthlyInterest,
periodTotal_)) /
    (pow(1 + monthlyInterest, periodTotal_) - 1);

}
```

Bug Fix: Input Validation

We added strict validation logic to prevent the processing of negative numbers for the Loan Amount, Interest Rate, or Period, which previously caused calculation errors

Code snippet

Main.cpp

// Validation logic to prevent negative inputs

```
if (amount < 0) {  
    cerr << "Error: Amount cannot be negative." << endl;  
    return CALC_UNKNOWN;  
}  
  
if (interest < 0) {  
    cerr << "Error: Interest cannot be negative." << endl;  
    return CALC_UNKNOWN;  
}  
  
if (period < 0) {  
    cerr << "Error: Period cannot be negative." << endl;  
    return CALC_UNKNOWN;  
}
```

Feature: Configuration File

We implemented a file reader that parses **config.txt** to load default values if the user does not provide command-line arguments.

Code snippet

Main.cpp

```

void readConfigFile(LoanCalculator &calculator) {
    ifstream file("config.txt");
    if (!file.is_open()) return;

    string line;
    while (getline(file, line)) {
        stringstream ss(line);
        string key;
        double value;

        // Parses "Key: Value" format
        if (getline(ss, key, ':') && ss >> value) {
            if (key.find("Amount") != string::npos)
calculator.setAmount(value);

            else if (key.find("Interest") != string::npos)
calculator.setInterest(value);

            else if (key.find("Period") != string::npos)
calculator.setPeriodTotal((int)value);
        }
    }
}

```

3. Unit Testing

We utilized the **GoogleTest** framework to ensure the reliability of the calculator. We implemented three specific test cases:

- **Standard EMI:** Verifies the calculation against a known correct value.

- **Large Tenure (Stress Test):** Tests the system's ability to handle a 50-year loan without overflowing.
- **Invalid Input:** Ensures the system handles a zero-period scenario gracefully

Test file snippet

Test_emi.cpp

```
TEST(LoanTest, LargeTenureLoan) {

    LoanCalculator loan;

    loan.setAmount(500000);

    loan.setInterest(3.5);

    loan.setPeriodTotal(600); // 50 years

    double emi = loan.calculatePayment();

    // Expecting 1766.00 with a small floating-point tolerance

    EXPECT_NEAR(emi, 1766.00, 0.1);

}
```

Test Execution Results

The screenshot below demonstrates that all 3 tests passed successfully.

Testing Ss

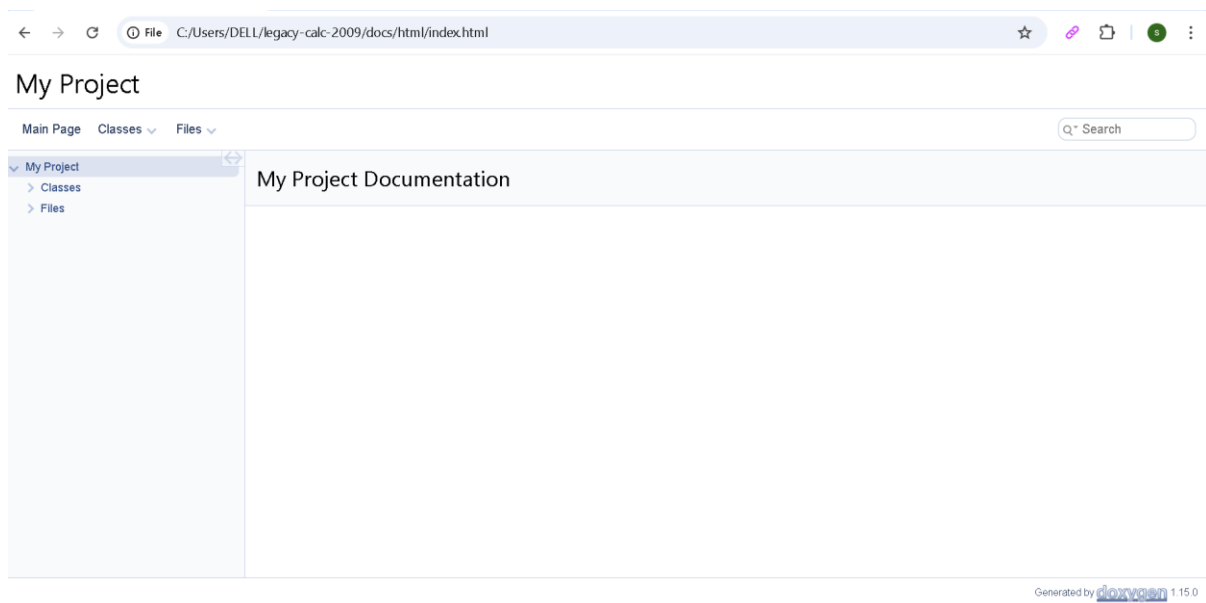
```
PS C:\Users\DELL\legacy-calc-2009> g++ -std=c++17 -isystem googletest/googletest/include -Igoogletest/googletest -pthread
src/Loan.cpp test/test_emi.cpp googletest/googletest/src/gtest-all.cc -o run_tests
PS C:\Users\DELL\legacy-calc-2009> ./run_tests.exe
[=====] Running 3 tests from 1 test suite.
[-----] Global test environment set-up.
[-----] 3 tests from LoanTest
[ RUN      ] LoanTest.CalculateStandardEMI
[ OK       ] LoanTest.CalculateStandardEMI (0 ms)
[ RUN      ] LoanTest.LargeTenureLoan
[ OK       ] LoanTest.LargeTenureLoan (0 ms)
[ RUN      ] LoanTest.ZeroPeriodHandle
[ OK       ] LoanTest.ZeroPeriodHandle (0 ms)
[-----] 3 tests from LoanTest (8 ms total)
[-----] Global test environment tear-down
[=====] 3 tests from 1 test suite ran. (17 ms total)
[ PASSED  ] 3 tests.
PS C:\Users\DELL\legacy-calc-2009> |
```

4. Documentation

We used **Doxygen** to generate professional code documentation. We annotated the **Loan.h** header file with Javadoc-style comments (@brief, @param, @return) to describe the purpose of every class and function.

Generated Documentation

The screenshot below shows the HTML index page generated by Doxygen, confirming the successful documentation of the `LoanCalculator` class.



5. Version Control History

We used Git for version control throughout the development process, creating a separate dev branch for features and committing regularly.